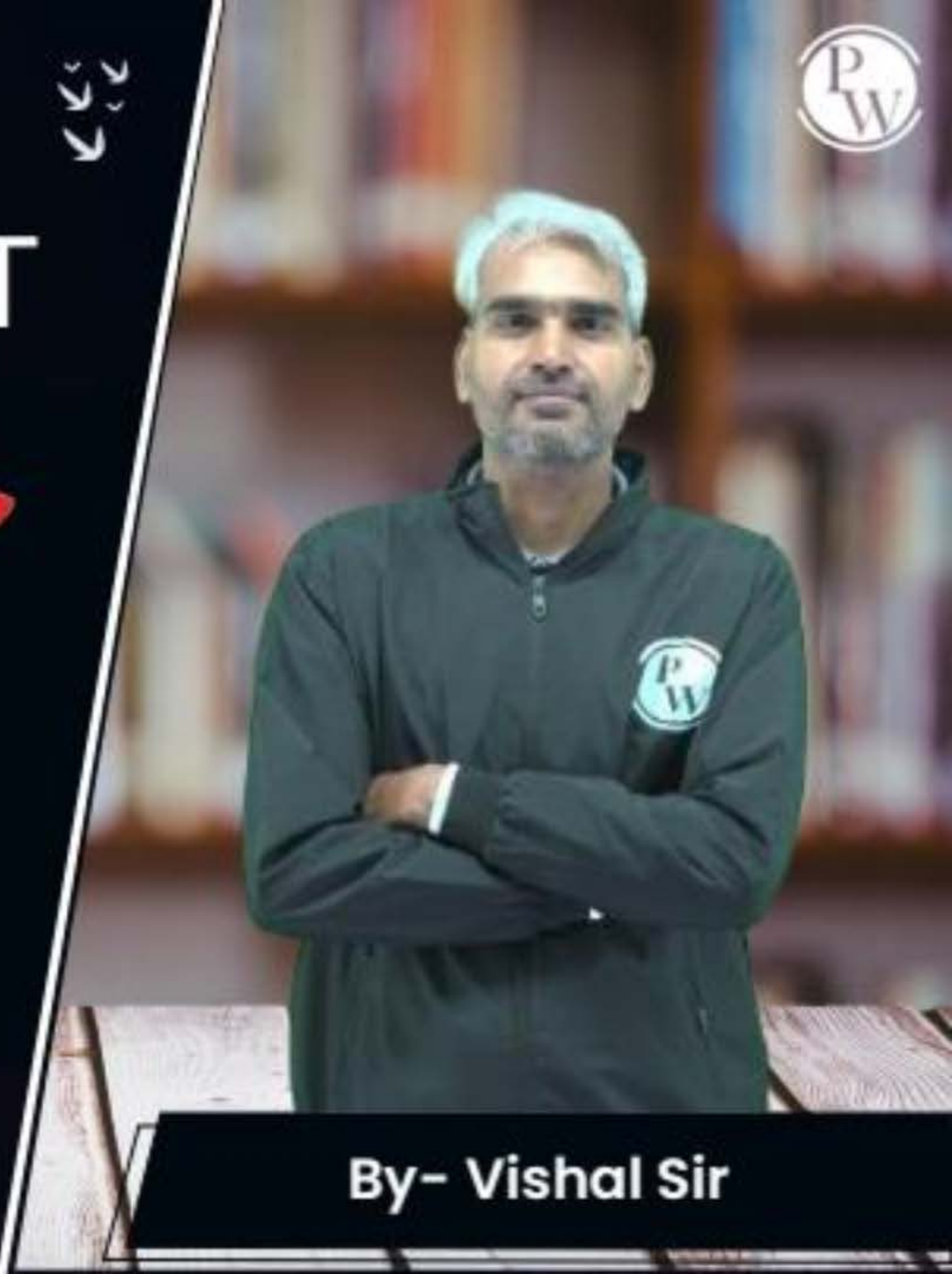
Computer Science & IT



Compiler Design

Syntax Analysis (Parser)

Lecture No. 02



By- Vishal Sir

Recap of Previous Lecture



Topic — Questions of Lexical Analysis

Topic — Introduction to Syntax Analyzer

Slide

Topics to be Covered

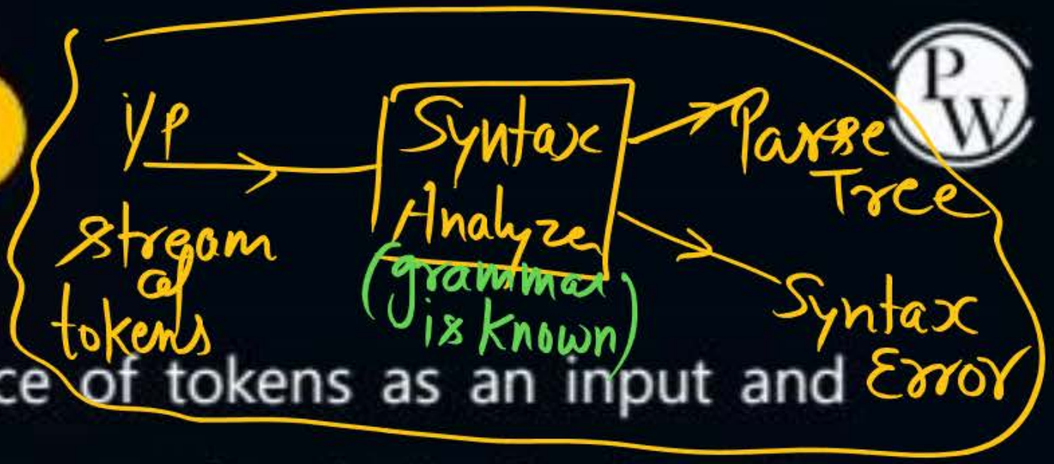


- Topic: Grammar and Types of grammar
- Topic: Types of Derivation Tree
- Topic: Types of Parser (Syntax Analyzer)
- Topic: Elimination of Left Recursion
- Topic: Left Factoring

Slide



Topic : Syntax Analyzer (Parser)



Syntax analyzer is a program that takes sequence of tokens as an input and tries to verify whether the sequence of tokens can be derived using the grammar known to the syntax analyzer or not.

If sequence of token (input string) is derivable from the grammar i.e., input string is member of language generated by grammar, then syntax analyzer produces the parse tree as an output otherwise it returns syntax error to the error handler.



Topic : Grammar

→ Grammar is a finite set of rules which are used to generate sentences corresponding to the language represented by that grammar

→ Grammar 'G' is defined using four tuples

$$G = (V, T, P, S)$$

- V = Set of Variables / Non-terminals (denoted using Capital letters)
- T = Set of terminals (denoted using lower case letters)
- P = Set of all production rules
- S = Start symbol

Slide

* Set of all strings (sentences) generated by the grammar defines the language generated by that grammar.



Topic : Grammar

Start Symbol

eg Grammar $G_1 = \{$

$S \longrightarrow aA / Bb$

$A \longrightarrow aB / b$

$B \longrightarrow C / \epsilon$

Non-terminals

$G_1 = (V, T, P, S)$

$V = \{S, A, B\}$

$T = \{a, b, C, \epsilon\}$

$P = \{ S \to aA,\ S \to Bb,\ A \to aB,\ A \to b,\ B \to C,\ B \to \epsilon \}$

$S = S$



Language of the above gramman is = {aac, aa, ab, cb, b}

H.W

Consider the following grammar,

$S \rightarrow aA$

$A \rightarrow bB / a$

$B \rightarrow Cd / \epsilon$

$C \rightarrow cCd / ddc$

find the language generated by the grammar.



Topic : Types of Grammar

Types of grammar

- Based on type of productions
  - Type-0
  - Type-1
  - Type-2
  - Type-3
- Based on number of derivation Tree
  - Ambiguous grammar
  - Unambiguous grammar
- Based on number of strings in the Language
  - Recursive grammar
  - {Non-Recursive grammar

Slide



Topic : Types of Grammar

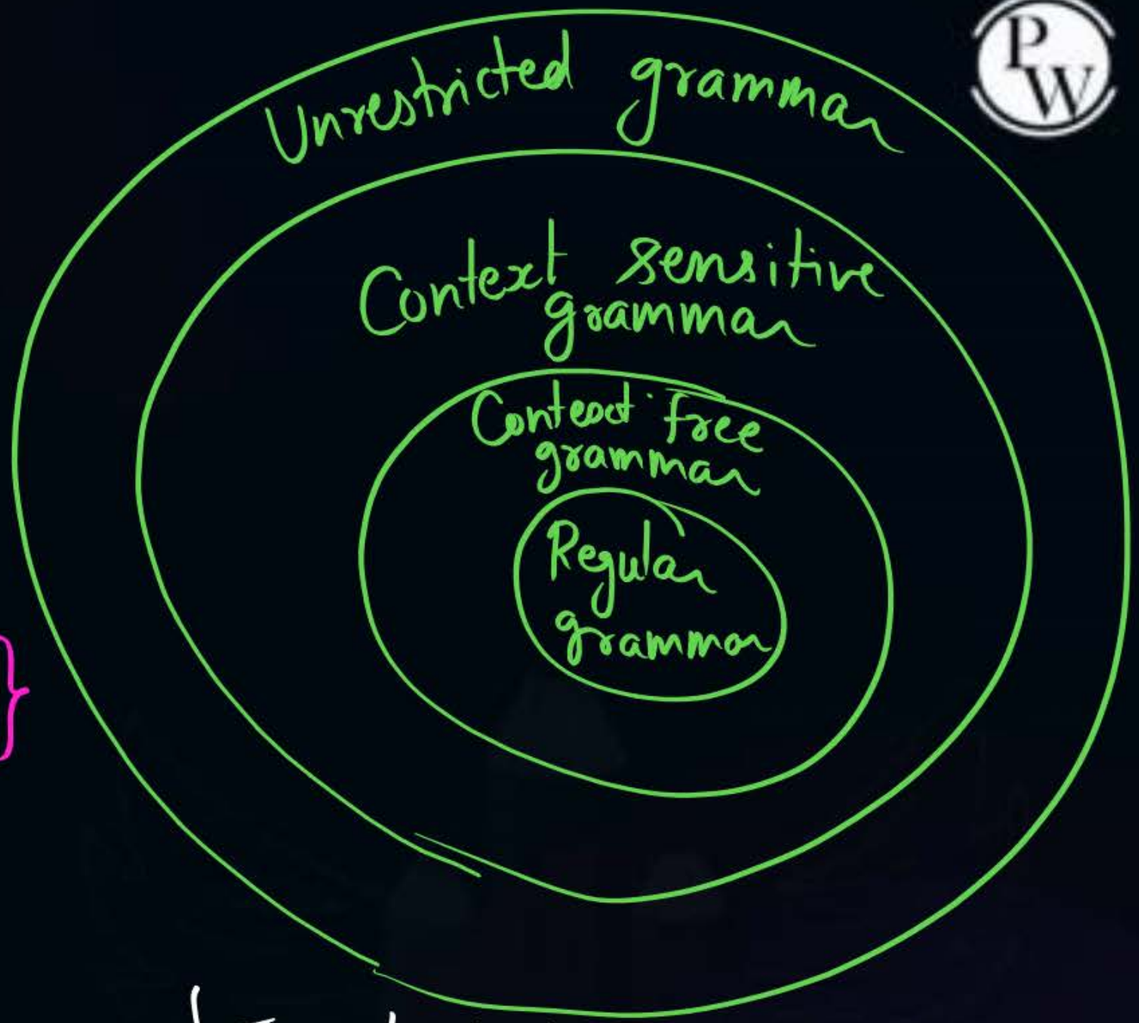
Based on type of productions :

Type - 0 {unrestricted grammar}

Type - 1 {Context sensitive grammar}

Type - 2 {Context free grammar}

Type - 3 {Regular grammar}



{To check the syntax (structure) the most suitable grammar is Context free grammar}

Slide

* Syntax analysis phase uses Context free grammar to check the syntax.

Topic : Types of Grammar

Knowledge is Not required W.r.t. Compiler design

Every Reg. grammar is CFG
∴ Both are allowed in CFG

Based on type of productions :

Regular grammar Type-3	Context free Type-2	Type-1	Type-0
$A \to xB/y$ or $A \to Bx/y$	$A \to \alpha/\beta$	$\alpha \to \beta$ where $\lvert\alpha\rvert \le \lvert\beta\rvert$	No restriction

Single non-terminal on LHS

x & y can be any combination of terminals

exactly one Non-terminal on LHS

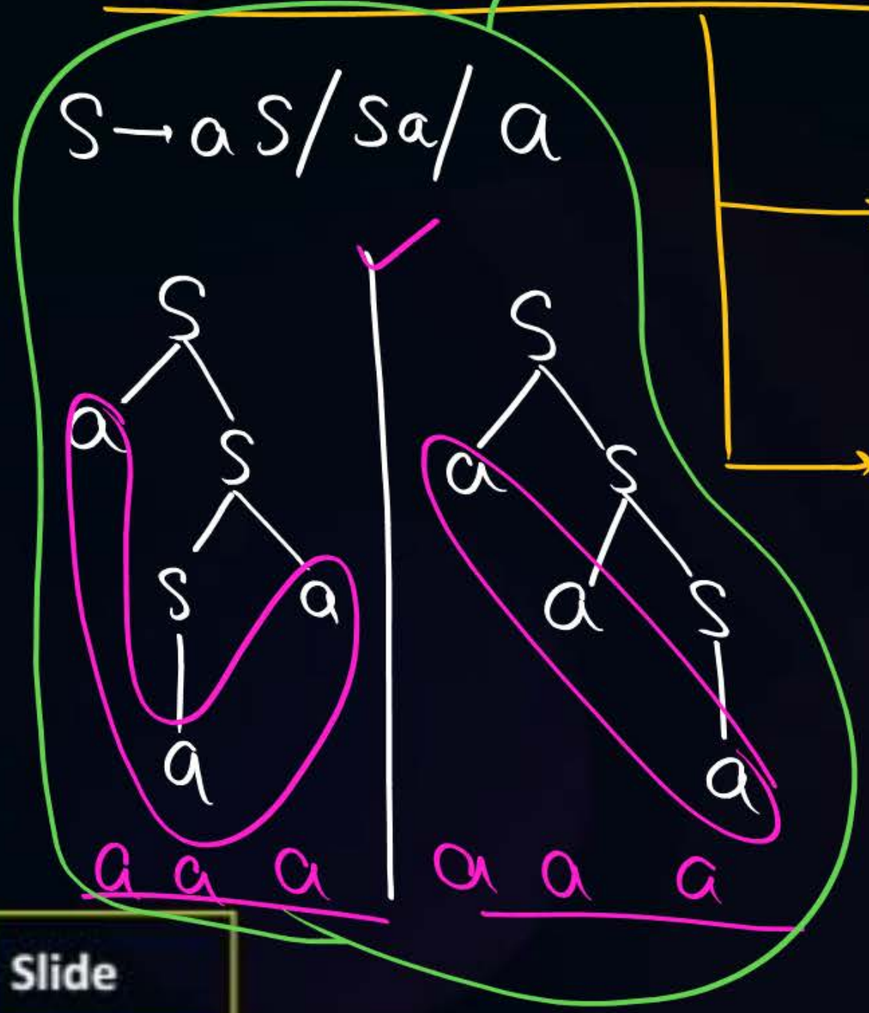
α β can be any combination of Variable & terminals

at most one non-terminal in RHS.

Slide

Topic : Types of Grammar

Based on number of derivation tree :



→ Ambiguous grammar

for string "aaa" these exists two or more distinct derivation tree ∴ grammar is ambiguous grammar

for any string in the language of the grammar if these exists more than one derivation tree (i.e distinct derivation trees) then grammar is called Ambiguous grammar.

→ Un-ambiguous grammar

For all the strings in the language generated by grammar if these exists exactly one derivation tree then grammar is called Unambiguous grammar

Slide



Topic : Derivation Tree

* The process of deriving the given input string, using the production rules of the grammar is called derivation, and its graphical representation is called derivation tree.

Derivation may be,

1. Leftmost derivation
2. Rightmost derivation
3. Random derivation

In parsing (Syntax Analysis) we will use either leftmost derivation or Rightmost derivation

Slide



Topic : Types of Derivation Tree

Leftmost derivation tree : During the derivation if leftmost non-terminal is expended first at step of derivation then it is called leftmost derivation

Slide



Topic : Types of Derivation Tree

Rightmost derivation tree : During the derivation if rightmost non-terminal is expended first at each step of derivation then it is called rightmost derivation

In Random derivation the non-terminals can be expended in any order.

Slide



Topic : Types of Grammar

Based on number of strings :

- Recursive grammar
  - The language generated by the grammar is infinite.
  - i.e. infinite number of strings exists in the language generated by grammar
- Non-Recursive grammar
  - The language generated by the grammar is finite, i.e. finite number of strings exists in the language generated by grammar

Slide



Topic : Types of Grammar

Based on number of strings :

Recursive gramma :-

$S \rightarrow aA / b$

$A \rightarrow abA / \epsilon B$

A produces A

$\therefore$ Recursive production

$B \rightarrow d / \epsilon$

if there exists any recursive production in the gramma then it is called Recursive gramma

But Sometimes Recursion May be indirect Recursion

Slide

Indirect Recursion

$S \rightarrow aA / b$

$A \rightarrow aBc / d$

$B \rightarrow bA / d$

No direct recursive production, but Indirect recursion exists

∴ Recursive grammar

Non-Recursive grammar :-

→ If no direct recursive production or no indirect recursion exists, then gramman is called non-recursive gramman, and language generated by gramman will contain finite number of strings in it.



2 mins Summary

Topic Different Types of grammar

Topic Types of Derivation Tree

Topic Types of Parser

Topic Elimination of Left Recursion

Topic Left Factoring

Slide

THANK - YOU

Slide